



PRODUCT REQUIREMENTS DOCUMENT (PRD)



Project Name

LaneIQ: Reinforcement Learning for Real-Time Highway Lane Optimization

1. Product Vision

Build a **realistic traffic simulation system** where an AI agent learns to make optimal lane-changing decisions under uncertainty, balancing:

- speed
- safety
- stability

The system should demonstrate:

- **deep technical ML understanding (RL)**
 - **systems design (simulation + backend)**
 - **visual storytelling (frontend demo)**
-

2. Problem Statement

Drivers constantly face:

"Should I change lanes or stay?"

This is a **sequential decision-making problem under uncertainty**, involving:

- incomplete information
- dynamic environments
- safety constraints

We aim to solve this using **Reinforcement Learning**, instead of heuristics.

3. 🧑🔧 Target Audience

- SWE / ML recruiters at companies like Google and Amazon
- Engineers reviewing your GitHub
- Yourself (learning RL deeply)

4. 🧩 Core Features

4.1 Traffic Simulation Engine

Requirements:

- Multi-lane highway (3–5 lanes)
- Vehicles with:
 - speed
 - position
 - acceleration
 - behavior profile:
 - ◆ aggressive
 - ◆ normal
 - ◆ slow

Dynamics:

- Random spawning
- Lane changes from other vehicles
- Speed variability
- Traffic density changes over time

4.2 RL Agent (Core of Project)

State Space

Include:

- ego vehicle speed
- distances to cars ahead/behind (same + adjacent lanes)
- relative velocities
- lane index

- local traffic density
-

Action Space

Discrete:

- stay
 - change left
 - change right
-

Reward Function (CRITICAL)

You will iterate on this:

Positive rewards

- maintaining high speed
- smooth driving

Negative rewards

- collisions (very high penalty)
 - unsafe distances
 - excessive lane switching
 - abrupt deceleration
-

Algorithms (Use Both)

- Start: Deep Q-Network (DQN)
- Upgrade: Proximal Policy Optimization (PPO)

Use:

- PyTorch
 - Stable-Baselines3
-

4.3 Baseline Systems (IMPORTANT)

You must compare against:

1. **Greedy heuristic**
 - pick lane with highest average speed
2. **Rule-based safe driver**
 - only change if clear + faster
3. **Random agent**

👉 This is what makes your project *credible*

4.4 Metrics & Evaluation System

Track:

Performance

- average speed
- time to destination

Safety

- collision rate
- near-miss frequency

Behavior Quality

- lane switches per minute
- acceleration smoothness

RL Metrics

- reward convergence
 - training stability
-

4.5 Visualization System (Huge Differentiator)

Must Have:

- moving cars in lanes (top-down view)
- highlight AI-controlled vehicle
- show decisions in real-time:
 - "Switching left due to faster lane"

Bonus:

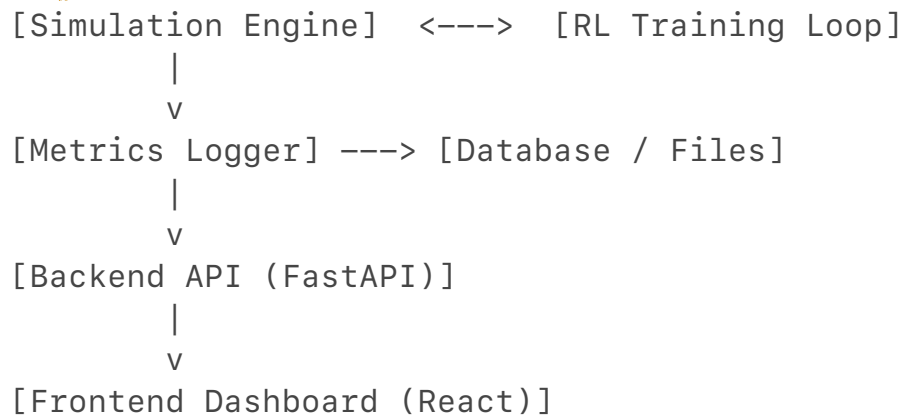
- heatmaps of lane efficiency
- graphs updating live

4.6 Web Demo (Polish Layer)

Stack:

- Backend: Python (FastAPI)
 - Frontend: React + TypeScript
 - Visualization: Canvas / WebGL
-

5. 🏗️ System Architecture



6. 🧪 Experimentation Plan

You'll run experiments like:

- RL vs heuristic at low traffic
 - RL vs heuristic at high traffic
 - effect of reward tuning
 - safety vs speed tradeoff
-

7. 📅 Milestones (1–2 Month Plan)

Week 1–2

- basic simulation (cars, lanes)

- simple rule-based agent

Week 3

- RL environment setup (Gym-style)
- implement DQN

Week 4

- train + debug agent
- add metrics logging

Week 5

- implement PPO
- compare results

Week 6–7

- visualization system
- frontend dashboard

Week 8

- polish + documentation
- record demo video

8. ⚠️ Risks (And How to Avoid Them)

Risk 1: RL doesn't converge

→ Keep environment simple first

Risk 2: Project becomes too big

→ Build vertical slices:

- simulation → RL → metrics → UI

Risk 3: Looks like a toy

→ fix with:

- baselines
- metrics
- clean visualization

9. 🚀 Stretch Goals (If You Have Time)

- multi-agent RL (many AI drivers)
- imitation learning from human policy
- uncertainty modeling (noisy sensors)

- integrate with SUMO (Simulation of Urban MObility)
-

10. 📄 Resume Bullets (What This Turns Into)

You'll eventually write something like:

- Built a reinforcement learning system using PPO and DQN to optimize real-time lane selection in a stochastic traffic simulation
 - Designed a multi-agent highway simulator modeling dynamic vehicle behavior and safety constraints
 - Improved average travel speed by X% over heuristic baselines while reducing unsafe events by Y%
 - Developed an interactive web-based visualization dashboard using React and FastAPI
-

11. 🎥 Demo Strategy (Underrated)

Your demo should show:

1. simulation running
 2. AI making decisions
 3. metrics improving vs baseline
-

12. 🧠 What You'll Learn (High Value)

- reinforcement learning deeply
 - system design
 - simulation modeling
 - frontend + backend integration
 - how to present technical work
-



Final Honest Advice

This project will only stand out if:

- your **RL actually works**
- you **compare against baselines**
- your **demo is clean and visual**

If you execute this well, it's *absolutely strong enough* for big tech internships.



Next Step

I can now break this into:

- **exact folder structure**
- **tech stack setup**
- **first 7 days step-by-step plan**
- **starter code outline**

Or we can zoom into the hardest part first:



RL setup vs simulation design

What do you want to tackle next?